

Data Preprocessing & Visualization

Session 3: (Matplotlib & Seaborn)

Agenda:

1	Introduction to Matplotlib
2	Introduction to Seaborn
3	Practical Visualization Examples

Introduction to Matplotlib

Introduction to Matplotlib

What is Matplotlib?

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. It is highly customizable and is widely used for producing high-quality 2D graphics.

It provides a MATLAB-like interface via the pyplot module, making it easy to create plots by calling a series of simple functions.

Key Features of Matplotlib

- **Line Plots:** Basic line plots for showing trends over time.
- **Scatter Plots:** For visualizing the relationship between two numerical variables.
- **Bar Charts:** Displaying categorical data with rectangular bars.
- **Histograms:** Showing the distribution of numerical data.
- **Subplots:** Creating multiple plots within a single figure.
- **Customization:** Highly customizable with titles, labels, legends, colors, and more

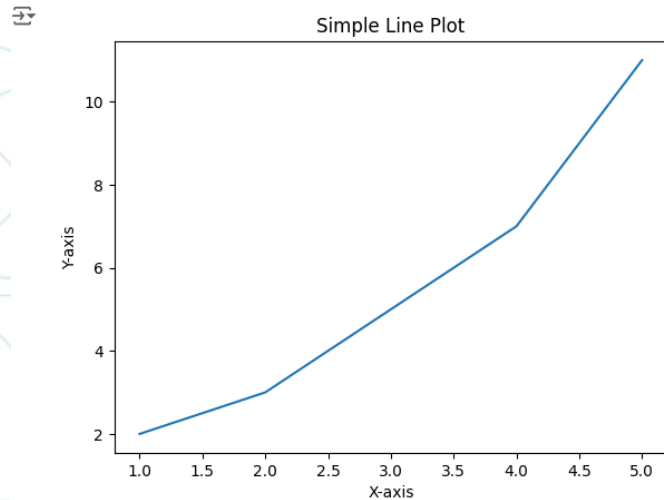
Basic Plotting with Matplotlib

Line Plot

```
import matplotlib.pyplot as plt

# Sample data
x = [1, 2, 3, 4, 5]
y = [2, 3, 5, 7, 11]

# Create a line plot
plt.plot(x, y)
plt.title("Simple Line Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```

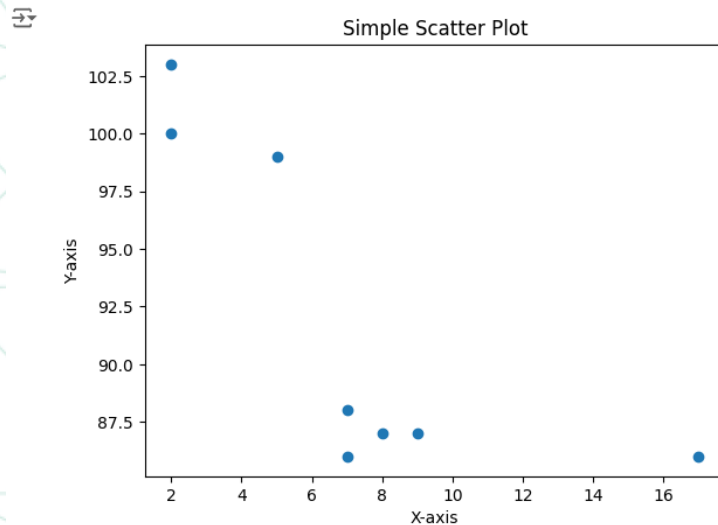


- This creates a basic line plot with x and y data points. `plt.title()`, `plt.xlabel()`, and `plt.ylabel()` add a title and labels to the axes.

Basic Plotting with Matplotlib

Scatter Plot

```
x = [5, 7, 8, 7, 2, 17, 2, 9]  
y = [99, 86, 87, 88, 100, 86, 103, 87]  
  
plt.scatter(x, y)  
plt.title("Simple Scatter Plot")  
plt.xlabel("X-axis")  
plt.ylabel("Y-axis")  
plt.show()
```

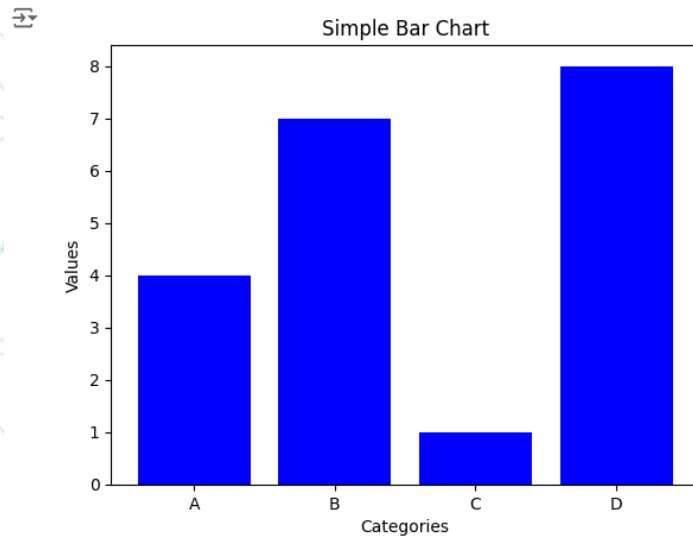


- A scatter plot is used to observe the relationship between two variables. `plt.scatter()` creates a scatter plot with x and y coordinates.

Basic Plotting with Matplotlib

Bar Chart

```
categories = ['A', 'B', 'C', 'D']  
values = [4, 7, 1, 8]  
  
plt.bar(categories, values, color='blue')  
plt.title("Simple Bar Chart")  
plt.xlabel("Categories")  
plt.ylabel("Values")  
plt.show()
```

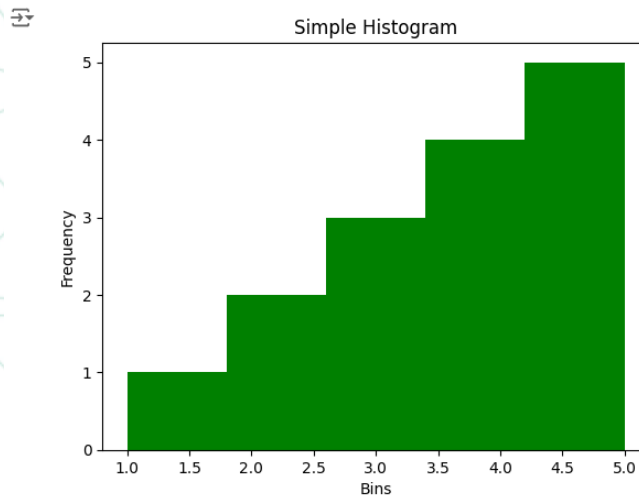


- A bar chart displays data with rectangular bars. `plt.bar()` is used to create a bar chart with specified categories and their corresponding values.

Basic Plotting with Matplotlib

Histogram

```
data = [1, 2, 2, 3, 3, 3, 4, 4, 4, 4, 5, 5, 5, 5, 5]  
  
plt.hist(data, bins=5, color='green')  
plt.title("Simple Histogram")  
plt.xlabel("Bins")  
plt.ylabel("Frequency")  
plt.show()
```



- A histogram is used to show the distribution of a numerical variable. `plt.hist()` creates a histogram with a specified number of bins.

Basic Plotting with Matplotlib

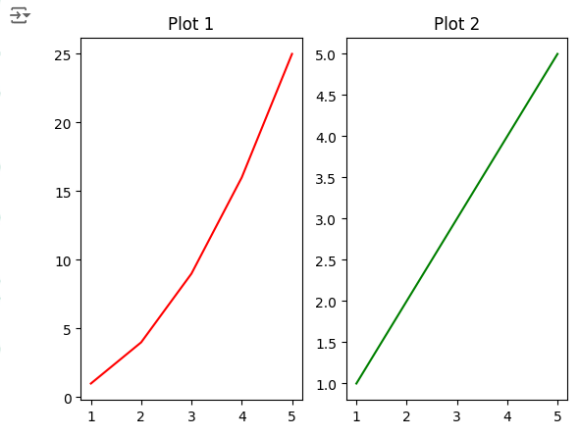
Subplots

```
x = [1, 2, 3, 4, 5]
y1 = [1, 4, 9, 16, 25]
y2 = [1, 2, 3, 4, 5]

plt.subplot(1, 2, 1)
plt.plot(x, y1, 'r-')
plt.title("Plot 1")

plt.subplot(1, 2, 2)
plt.plot(x, y2, 'g-')
plt.title("Plot 2")

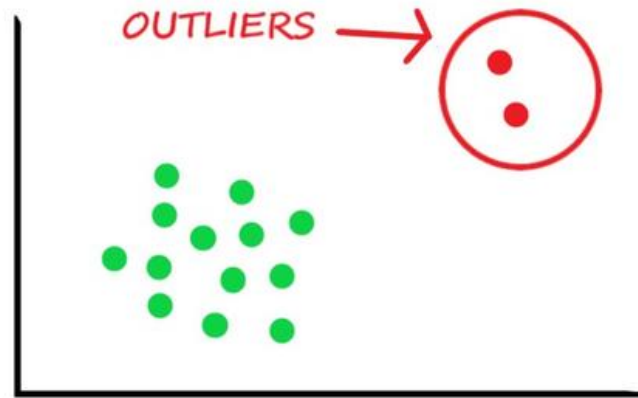
plt.show()
```



- Subplots allow multiple plots to be shown in a single figure.
`plt.subplot(1, 2, 1)` creates the first subplot, and `plt.subplot(1, 2, 2)` creates the second.

Outliers

Outliers are data points that significantly deviate from the rest of the data in a dataset. These anomalies can occur due to inconsistencies in measurement, errors during data collection, or actual variations. Outliers have the potential to distort statistical results and negatively influence machine learning models. Hence, it's crucial to detect and manage them effectively.



Identifying Outliers

- **Visualization Methods:** Techniques such as box plots, scatter plots, histograms, and density plots are valuable tools for detecting outliers visually.
- **Box Plot:** Displays data distribution, highlighting outliers as individual points outside the whiskers.
- **Scatter Plot:** Shows relationships between variables, where outliers can be seen as points far from the main cluster.
- **Histogram:** Illustrates the frequency of data points, with outliers appearing as bars that are distant from the rest.
- **Density Plot:** Provides a smooth curve of the data distribution, with outliers showing up as irregular bumps outside the main peak.

Introduction to Seaborn

Introduction to Seaborn

What is Seaborn?

Seaborn is a Python data visualization library based on Matplotlib. It provides a high-level interface for drawing attractive and informative statistical graphics. Seaborn makes it easy to create complex visualizations, including relationships, categorical data, and statistical plots.

Key Features of Seaborn

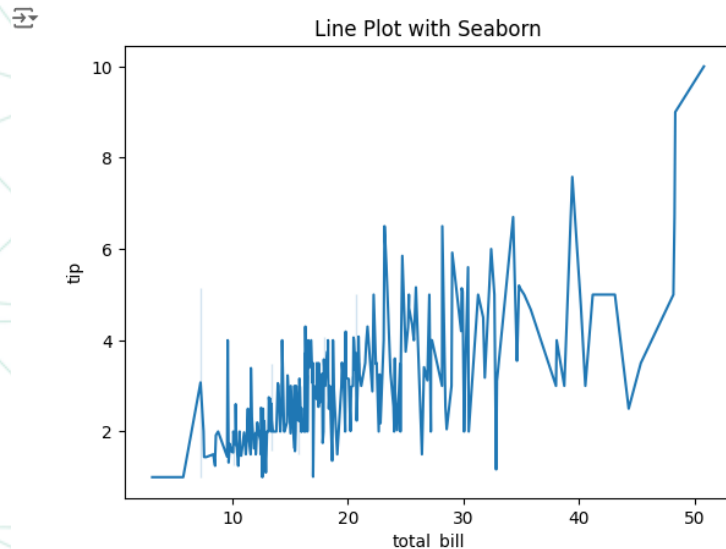
- **Visualizing Distributions:** Histograms, KDE plots, and rug plots.
- **Visualizing Categorical Data:** Bar plots, count plots, box plots, and violin plots.
- **Visualizing Relationships:** Scatter plots, line plots, and pair plots.
- **Heatmaps:** For showing data correlations.
- **Themes:** Built-in themes for better-looking plots.

Basic Plotting with Seaborn

Line Plot

```
import seaborn as sns
import matplotlib.pyplot as plt

data = sns.load_dataset('tips')
sns.lineplot(x='total_bill', y='tip', data=data)
plt.title("Line Plot with Seaborn")
plt.show()
```

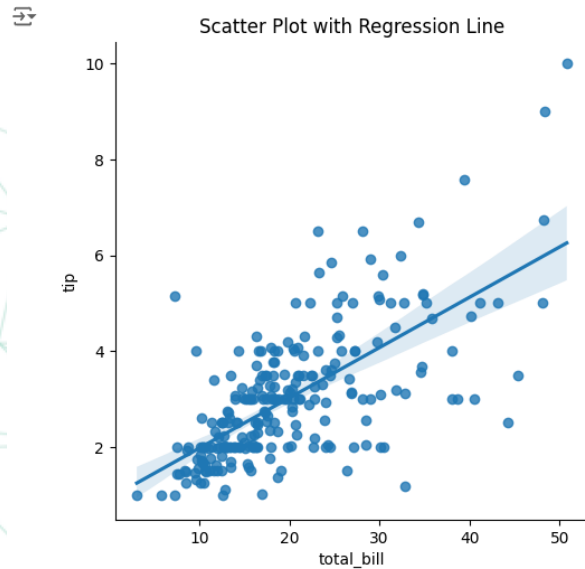


- This uses Seaborn's lineplot() to create a line plot using the 'tips' dataset, showing the relationship between total bill and tip.

Basic Plotting with Seaborn

Scatter Plot with Regression Line

```
sns.lmplot(x='total_bill', y='tip', data=data)  
plt.title("Scatter Plot with Regression Line")  
plt.show()
```

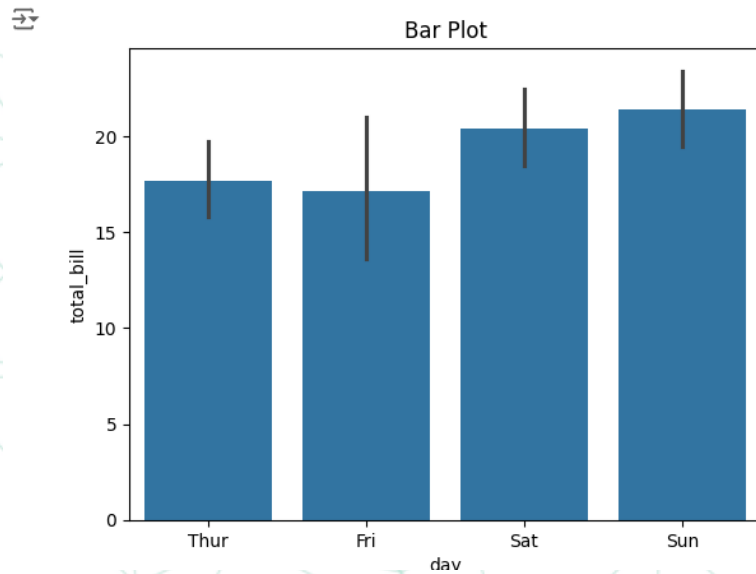


- Seaborn's `lmplot()` creates a scatter plot with a linear regression line, illustrating the trend between total bill and tip.

Basic Plotting with Seaborn

Bar Plot

```
sns.barplot(x='day', y='total_bill', data=data)  
plt.title("Bar Plot")  
plt.show()
```

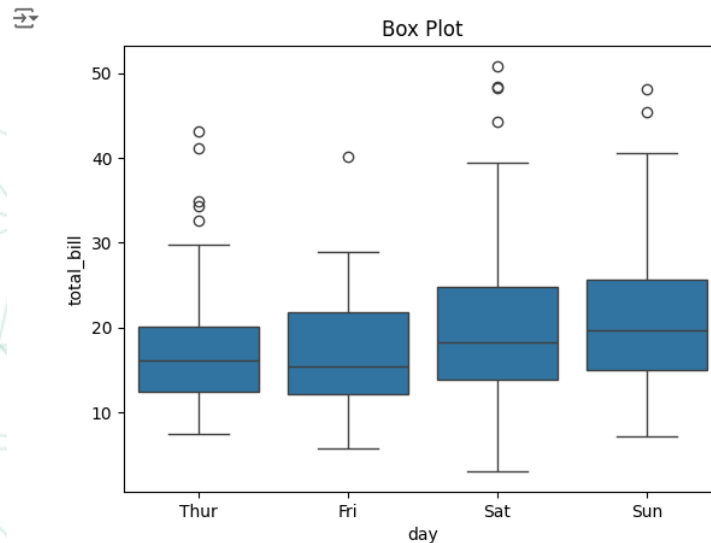


- `barplot()` creates a bar chart showing the average total bill for each day of the week. It aggregates the data automatically.

Basic Plotting with Seaborn

Box Plot

```
sns.boxplot(x='day', y='total_bill', data=data)  
plt.title("Box Plot")  
plt.show()
```



- `boxplot()` is used to show the distribution of total bills across different days, highlighting the median, quartiles, and potential outliers.

Basic Plotting with Seaborn

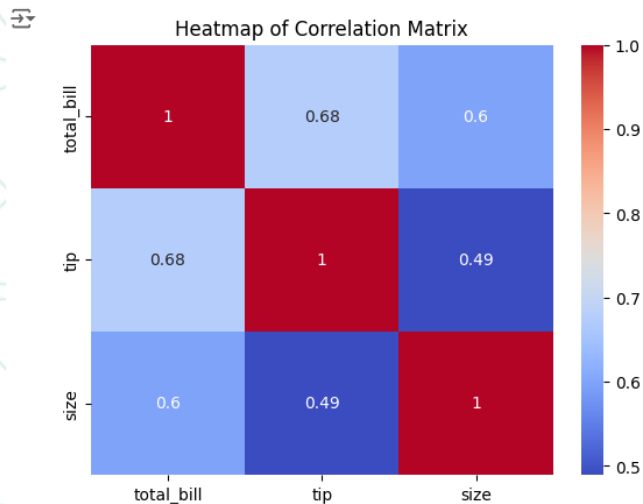
Heatmap

```
# Load a dataset using Seaborn
data = sns.load_dataset('tips') # 'tips' is a built-in dataset in Seaborn

# Select only the numerical columns
numerical_data = data.select_dtypes(include=['float64', 'int64'])

# Compute the correlation matrix
corr = numerical_data.corr()

# Plot the heatmap
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title("Heatmap of Correlation Matrix")
plt.show()
```



- `heatmap()` visualizes the correlation matrix of the dataset, showing relationships between variables using color gradients.

Practical Visualization Examples

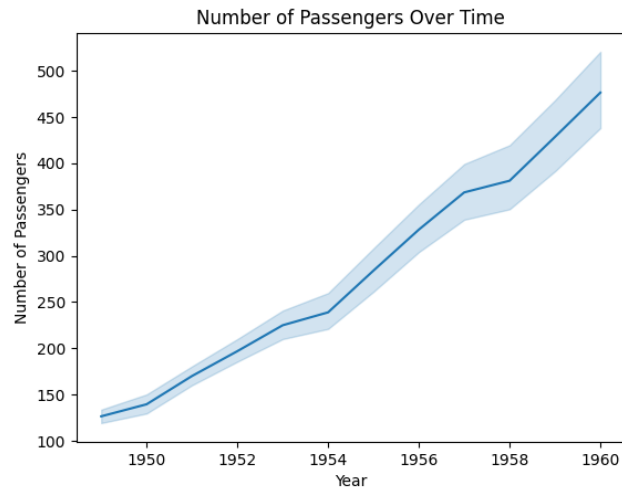
Practical Visualization Examples

Line Plot to Show Trends Over Time

```
[14] import seaborn as sns
import matplotlib.pyplot as plt

# Load the 'flights' dataset
data = sns.load_dataset('flights')

# Line plot showing the number of passengers over time
sns.lineplot(x='year', y='passengers', data=data)
plt.title("Number of Passengers Over Time")
plt.xlabel("Year")
plt.ylabel("Number of Passengers")
plt.show()
```



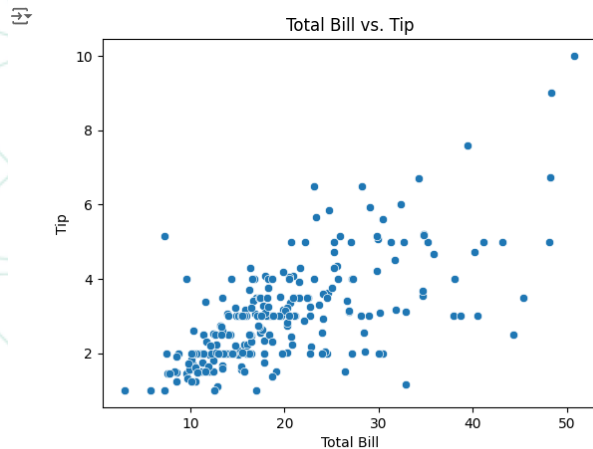
- This line plot shows the trend in the number of passengers over time using the 'flights' dataset. It helps to identify overall growth or decline trends.

Practical Visualization Examples

Scatter Plot to Visualize Relationships Between Two Variables

```
# Load the 'tips' dataset
data = sns.load_dataset('tips')

# Scatter plot showing the relationship between total bill and tip
sns.scatterplot(x='total_bill', y='tip', data=data)
plt.title("Total Bill vs. Tip")
plt.xlabel("Total Bill")
plt.ylabel("Tip")
plt.show()
```



- A scatter plot shows the relationship between the total bill and the tip amount in the 'tips' dataset. It helps in identifying patterns or correlations.

Practical Visualization Examples

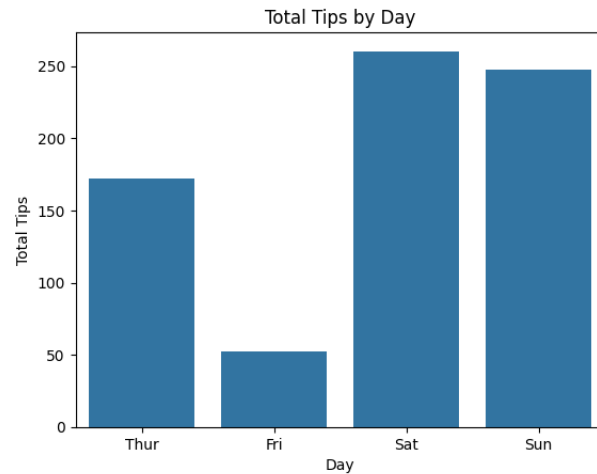
Bar Plot to Show Categorical Data

```
# Bar plot showing the average tip by day
sns.barplot(x='day', y='tip', data=data, estimator=sum, ci=None)
plt.title("Total Tips by Day")
plt.xlabel("Day")
plt.ylabel("Total Tips")
plt.show()
```

<ipython-input-16-e892f68ee143>:2: FutureWarning:

The `ci` parameter is deprecated. Use `errorbar=None` for the same effect.

```
sns.barplot(x='day', y='tip', data=data, estimator=sum, ci=None)
```

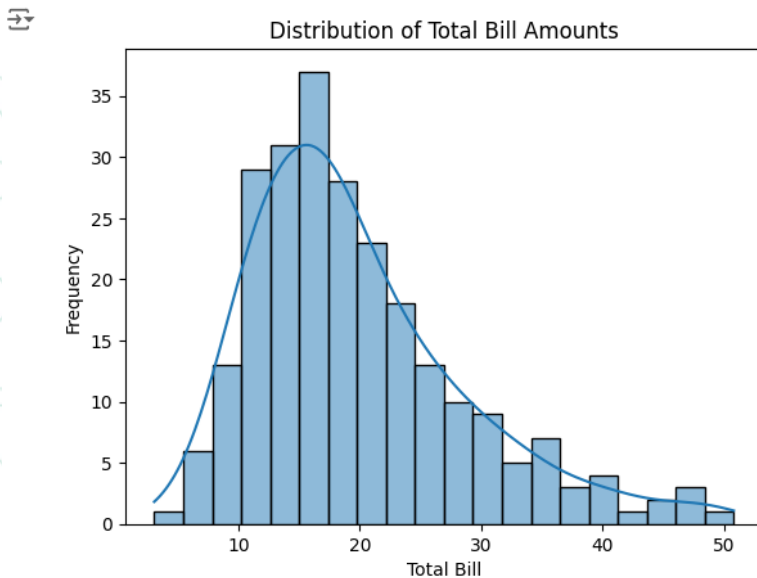


- This bar plot shows the sum of tips collected for each day. It's useful for comparing categorical data and understanding how different days perform in terms of tip collection.

Practical Visualization Examples

Histogram to Show Data Distribution

```
# Histogram of total bill amounts
sns.histplot(data['total_bill'], bins=20, kde=True)
plt.title("Distribution of Total Bill Amounts")
plt.xlabel("Total Bill")
plt.ylabel("Frequency")
plt.show()
```

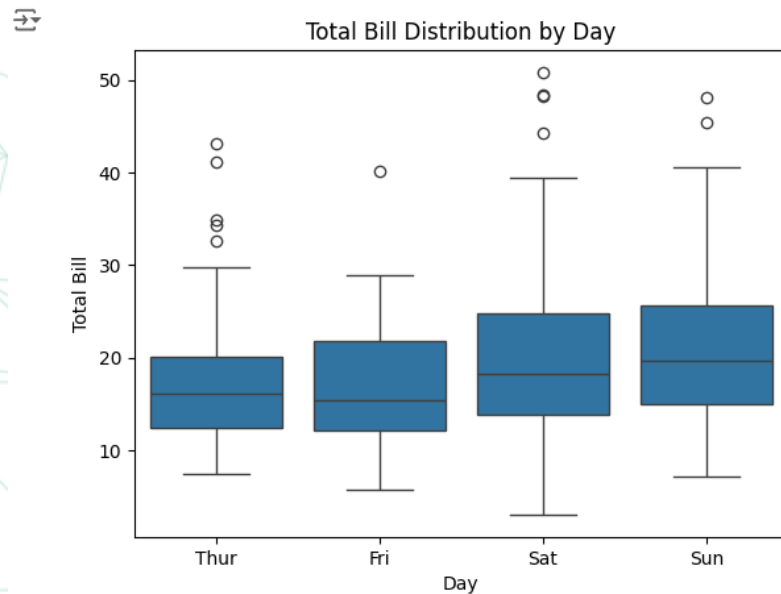


- This histogram displays the distribution of total bill amounts. The KDE (Kernel Density Estimate) line provides a smoothed version of the distribution, making it easier to see the underlying pattern.

Practical Visualization Examples

Box Plot to Visualize Distribution and Outliers

```
# Box plot showing the distribution of total bill amounts by day
sns.boxplot(x='day', y='total_bill', data=data)
plt.title("Total Bill Distribution by Day")
plt.xlabel("Day")
plt.ylabel("Total Bill")
plt.show()
```

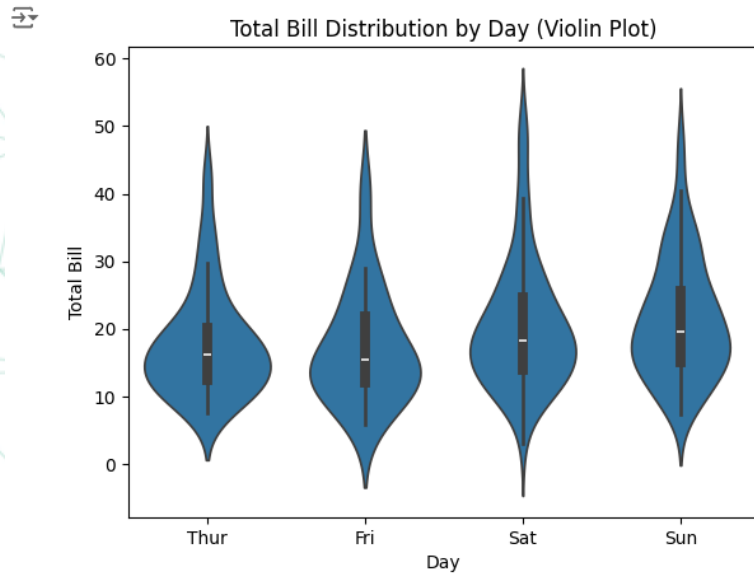


- Box plots provide a summary of the distribution, including median, quartiles, and potential outliers, for the total bill amounts categorized by day.

Practical Visualization Examples

Violin Plot to Show Density and Distribution

```
# Violin plot showing the distribution of total bill amounts by day  
sns.violinplot(x='day', y='total_bill', data=data)  
plt.title("Total Bill Distribution by Day (Violin Plot)")  
plt.xlabel("Day")  
plt.ylabel("Total Bill")  
plt.show()
```

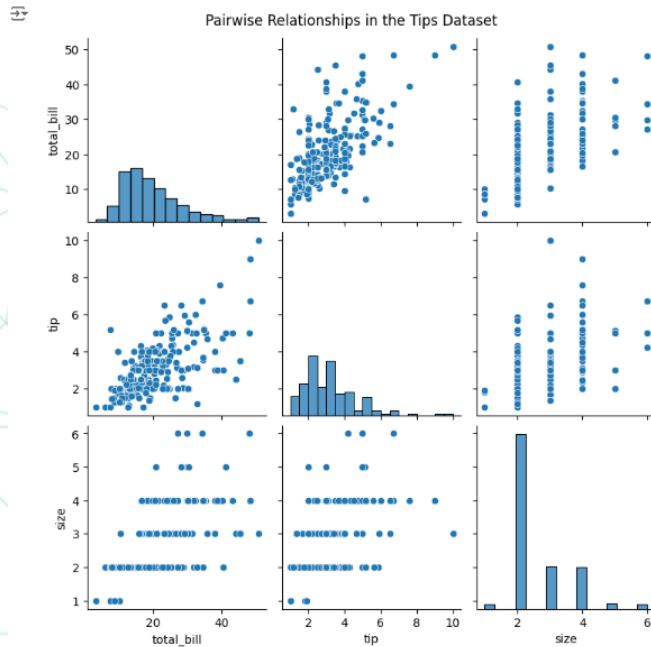


- Violin plots combine the features of box plots and KDE, showing the distribution and density of total bill amounts by day.

Practical Visualization Examples

Pair Plot to Show Pairwise Relationships

```
# Pair plot showing pairwise relationships in the 'tips' dataset  
sns.pairplot(data)  
plt.suptitle("Pairwise Relationships in the Tips Dataset", y=1.02)  
plt.show()
```

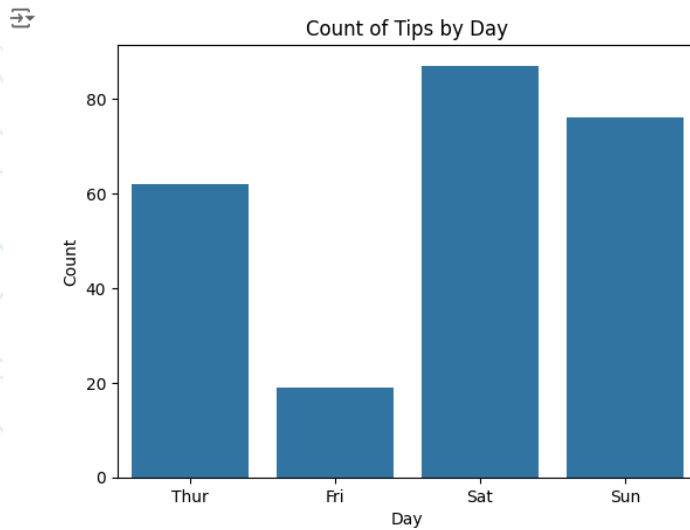


- Pair plots display pairwise relationships between different numerical variables in the dataset, helping to identify potential correlations and distributions.

Practical Visualization Examples

Count Plot to Show the Frequency of Categorical Values

```
# Count plot showing the count of tips by day  
sns.countplot(x='day', data=data)  
plt.title("Count of Tips by Day")  
plt.xlabel("Day")  
plt.ylabel("Count")  
plt.show()
```

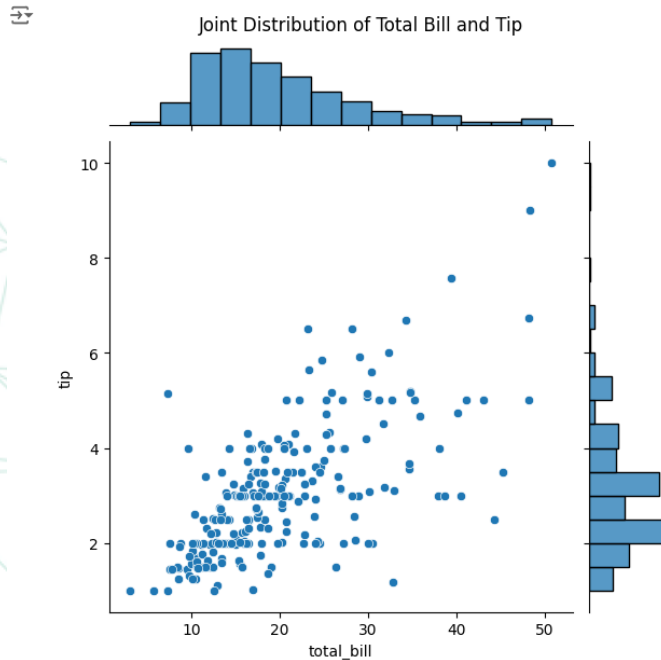


- Count plots visualize the frequency of occurrences for categorical data. This example shows how many entries exist for each day in the 'tips' dataset.

Practical Visualization Examples

Joint Plot to Show Distribution and Relationship

```
# Joint plot showing the distribution of total bill and tip amounts  
sns.jointplot(x='total_bill', y='tip', data=data, kind='scatter')  
plt.suptitle("Joint Distribution of Total Bill and Tip", y=1.02)  
plt.show()
```



- A joint plot combines a scatter plot and histograms, showing the relationship between total bill and tip amounts, along with their individual distributions.